Visual Studio Community .NET

C#

Visual Studio .NET

- Nuevo Proyecto
  - Nombre
  - Ubicación
  - Nombre solución
  - Esqueleto del nuevo proyecto (programa c#)

```
using System;                                                                    //
incluye espacios de nombres
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace CalculoSueldo                                                          //
agrupa clases y estructuras
{
        static class Program
        {
                                        static void Main()
                                        {
                                                                                 .....
                                                                                 Application.Run
        (new Form1());
                                }
         }
  }
```

  - Revisar carpeta con nuestro nuevo programa

Variables

- Empiezan por una letra.
- Pueden estar compuestas por caracteres alfanuméricos o el carácter subrayado “_”.
- Se distinguen mayúsculas de minúsculas
- No se pueden usar las palabras reservadas del lenguaje (if,switch, void, …) a no ser que se utilice el carácter arroba “**@**” precediendo al la variable. Por ejemplo.: **@if** sí podría ser una variable.

Visual Studio 2022, variables

- **Tipos de datos primitivos en C#**

ENTEROS: int – byte – sbyte – short – ushort – uint – long – ulong [byte v1 = 200; short v3 = 30000;]

REALES: float - double - decimal.

Char: un carácter Unicode char letra='A'

Cadena : string texto = "hola";

Lógico: bool encontrado = false;

"Mínimo:" + float.MinValue Valor minimo

"Máximo:" + float.MaxValue Valor máximo

sizeof(short) tamaño en bytes

Ej. Mostrar valor max, min para cada tipo de dato primitivo en C#

El lenguaje C# es fuertemente tipado (es decir que toda variable en tiempo de compilación tiene que identificar que tipo de , var x=10; El compilador interpreta que estamos definiendo una variable entera.

Variables

- **Tipos numéricos enteros:**

  Con signo

  sbyte (8 bits)
  short (16 bits)
  int (32 bits)
  long (64 bits)

  Sin signo

  byte (8 bits)
  ushort (16 bits)
  uint (32 bits)
  ulong (64 bits)

- **Tipos numéricos decimales** (todos son con signo).

  float (4 bytes = 32 bits)
  double (8 bytes = 64 bits)
  decimal (16 bytes = 128 bits)

- **Tipo Booleano**: **bool**
- **Tipos de carácter**:

  char (2 bytes = 16 bits): código unicode donde se pueden representar hasta 256 caracteres, de los cuales los primeros 128 son los mismos que el juego de caracteres ASCII.

  String: es el objeto que utilizamos para representar cadenas.

Variables, definición

- Declaración de variables

  Tipo nombreVar [=valor inicial][,nombreVar2]

  En caso de que se omita el valor inicial, la variable será inicializada a **cero** si corresponde a un tipo **numérico**; a una cadena de **carácter vacío** si es del tipo **String**; al valor **null** si es del tipo **Object**, y a **false** si es del tipo **bool**.

  En el interior de una función es necesario asignar valor inicial.

Visual Studio 2022

- Definición de variables

```
int var1;
float var2;
string linea;
Console.write(“Mensaje:”)
Console.write(var1);
linea = Console.readline();
Var1 = int.Parse(linea);
Var2 = float.Parse(linea);
```

Visual Studio 2022

- Errores sintácticos
  - console.
  - int var; ✉console.write(Var);

- Errores lógicos
  - Superficie = lado * lado * lado
- Comentarios // xxxxxx - /* texto */

Cadena String / char

- Char, representa un carácter UNICODE (16 bits)
- String, representa un texto como una serie de caracteres UNICODE

➢ Inicializar:

- Msg = null;
- Msg = string.Empty ✉“” (NullReferenceException)
- s1 = s2; s1 += s2
- S2[n] ✉char
- Ej. CajadeTexto.Text = “0”;

Cadenas ***

- Cadena.Clone()
- Cadena.CompareTo(valor) int
- Cadena.Contains(valor) bool
- Cadena.CopyTo (sourceIndex, destino, destinoIndex, num)
- Cadena.EndsWith(valor) bool
- Cadena.StartsWith (string) bool
- Cadena.Equals(valor) bool
- Cadena.IndexOf(char) int
- Cadena.LastIndexOf(char) int
- Cadena.Insert(starIndex, String)
- Cadena.Length
- Cadena.PadRight(n)
- Cadena.PadLeft(n)
- Cadena.Remove(Index, numcar)
- Cadena.Replace(char old, char new)
- Cadena.Substring(start,long)
- Cadena.ToLower()
- Cadena.ToUpper()
- Cadena.Trim(chars)
- Cadena.TrimEnd(chars)
- Cadena.TrimStart(chars)

Cadenas ***

- Ej. Uso de métodos
  - String.IsNullOrEmpty
  - String.IsNullOrWhitSpace
  - Char.IsDigit
  - Char.IsLetter …….

Visual Studio 2022- cadenas

Asignar valor a una cadena

- ```
  string cadena;
  cadena="ha dicho: \"¡Revolución!\"";
  Console.WriteLine(cadena);
  ```

Extracción de un carácter particular

- ```
  cadena1="el invierno será lluvioso";
  cadena2="el invierno será frío";
  Console.WriteLine("El cuarto caracter de la cadena1 es:{0}",cadena1[3]);
  ```

Longitud de la cadena

- ```
  Console.WriteLine("la cadena1 contiene {0} caracteres",cadena1.Lenght);
  //Que dará como resultado 25
  ```

División de una cadena

- ```
  Console.WriteLine("Un trozo de la cadena2= {0}", cadena2.Substring(2,8));
  //Que dara como resultado invierno
  ```

Visual Studio 2022- cadenas

- Inserción en una cadena

```
string cadena3;
cadena3 = cadena2.Insert(17,"muy ");
Console.WriteLine(cadena3);
//Resultado: EL invierno será muy frío
```

- Supresión de espacios

```
Cadena4 = cadena1.TrimStart();
Cadena4 = cadena1.TrimEnd();
Cadena4 = cadena1.Trim()
```

- Cambio a mayúsculas, minúsculas.

```
Cadena4 = cadena1.ToUpper();
Cadena4 = cadena1.ToLower();
```

Visual Studio 2022- cadenas

- Búsqueda en una cadena

```
string busca="@";
string cadena1="Si tiene algún problema contacta con info@aulaweb.com.es";
int posicion = 0;
posicion = cadena1.IndexOf(busca);
while (posicion&gt;0) {
   Console.WriteLine("cadena encontrada en la posicion {0}",posicion);
   posicion = cadena1.IndexOf(busca, posicion+1); }
Console.WriteLine("fin de la busqueda");
```

- Sustitución en una cadena

```
cadena3 = cadena1.Replace("invierno","verano");
Console.WriteLine(cadena3);
```

Fechas ***

- Representa un instante de tiempo, normalmente expresado en forma de fecha y hora del día.
- DateTime (int32 , int32 , int32) año, mes, día
- DateTime fecha = new DateTime(2008,5,1,8,30,52);año,mes,dia,hh,mm,ss
- DateTime fecha1 = DateTime.Now;
- DateTime fecha2 = DateTime.UtcNow; // horaUtc
- DateTime fecha3 = DateTime.Today;

- String dateString = "5/1/2008 8:30:52 AM";
- DateTime fecha = DateTime.Parse(dateString, System.Globalization.CultureInfo.InvariantCulture);

Fechas – DateTimePicker (1 Fecha)

- Obtiene o establece el valor de fecha y hora asignado al control
- dateTimePicker1.Value
- Evento dateTimePicker1_ValueChanged (….)

- Label1.Text = DateTime.Now.ToShortTimeString();

- DateTime fecha;
- fecha = dateTimePicker1.Value
- Fecha.addxxx(n) ‘ dias, segundos, años …..
- Fecha.ToLongTimeString(), ToLongDateString …..

Fechas – **MonthCalendar** (rango fechas)

- Por defecto fecha actual y aparece en la parte inferior.
- ShowToday . false
- ShowTodayCircle .false
- ShowWeekNumbers . True, nº semana
- CalendarDimensions
- FirstDayOfWeek
- BoldedDates, AnnuallyBoldedDates, MonthlyBoldedDates
- **SelectionRange rango de selección de fechas**
  - **DateTime > monthCalendar1.SelectionStart**
  - **DateTime > monthCalendar1.SelectionEnd**

  - **monthCalendar1.SelectionRange.Start**
  - **monthCalendar1.SelectionRange.End**
- **MaxSelectionCount, nº max de días a seleccionar**
- **MaxDate**
- **MinDate**

➢ **Objeto Timer (Prop. Interval // Evento _Tick) ‘ Control de tiempo**

Fechas - diferencias

- DateTime oldDate = new DateTime(2002,7,15);
  DateTime newDate = DateTime.Now;

```
// Difference in days, hours, and minutes.
TimeSpan ts = newDate - oldDate;

// Difference in days.
int differenceInDays = ts.Days;

System.Console.WriteLine("Difference in days: {0} ",
                                        differenceInDays);
System.Console.ReadLine();
```

Variables – conversión de tipos

- **Conversión de tipos:**

  Cuando convertimos de un tipo superior a otro inferior hay riesgo de perdida de información. Por ejemplo un tipo "**double**" a otro tipo "**long**". Para limitar este riesgo el compilador activa un error cuando se encuentra con esta situación. No obstante este tipo de conversión no está prohibida, para poder utilizarla simplemente hacemos una conversión explícita.

```
double x;
long y;
x = 21.234323;
y = (long) x;
Console.WriteLine ("valor de x:" + x);
Console.WriteLine ("valor de y:" + y);
```

Variable – conversión desde una cadena de caracteres

- De cadena a un tipo numérico

```
float iva = float.Parse(“21”);
```

- Si la conversión no se realiza correctamente se genera un excepción. Para evitar esto podemos utilizar la función TryParse.

```
if (float.TryParse("21", out iva)) {
    Console.writeLine("Conversión Ok."); }
else {
    Console.WriteLine("Problema durante la conversión"); }
```

Ámbito variables

- **A nivel de bloque:** Sólo el código del bloque tendrá la posibilidad de trabajar con la variable (por ejemplo un bucle ***for next***)
- **A nivel de función**: Sólo el código de la función donde se declara la variable podrá modificar su contenido, se llama normalmente variable local.
- **A nivel de clase**: Una variable declarada en el interior de una clase es accesible al código de esta clase sin restricción y eventualmente a partir de otras porciones de código en función del nivel de acceso de la variable.

- **public**: Los elementos declarados serán accesibles desde cualquier porción de código del proyecto y desde cualquier otro proyecto que haga referencia a aquel donde están declarados. No se pueden utilizar dentro en el interior de las funciones.
- **private**: restringe el acceso a la variable al módulo, a la clase o a la estructura en la cual está declarada. No se puede utilizar en el interior de un procedimiento o función.

Ámbito variables

- **public**: Los elementos declarados serán accesibles desde cualquier porción de código del proyecto y desde cualquier otro proyecto que haga referencia a aquel donde están declarados. No se pueden utilizar dentro en el interior de las funciones.
- **protected**: Se puede utilizar en el interior de una clase. Permite restringir el acceso a la variable únicamente al código de la clase y todas las clases que hereden de ella.
- **internal:** Serán accesibles desde el ensamblado en el cual están declarados y tampoco se pueden utilizar en el interior de una función.
- **protected internal:** Es el nivel de acceso de protected e internal.
- **private:** restringe el acceso a la variable al módulo, a la clase o a la estructura en la cual está declarada. No se puede utilizar en el interior de un procedimiento o función.

Visual Studio 2022

- **Problemas propuestos**
- Realizar la carga del lado de un cuadrado, mostrar por pantalla el perímetro del mismo (El perímetro de un cuadrado se calcula multiplicando el valor del lado por cuatro)
- Escribir un programa en el cual se pidan cuatro números, calcular e informar la suma de los dos primeros y el producto del tercero y el cuarto.
- Realizar un programa que lea cuatro valores numéricos e informar su suma y promedio.
- Se debe desarrollar un programa que pida el ingreso del precio de un artículo y la cantidad que lleva el cliente. Mostrar lo que debe abonar al comprador.

Visual Studio 2022

- Realizar un programa que lea por teclado dos números, si el primero es mayor al segundo informar su suma y diferencia, en caso contrario informar el producto y la división del primero respecto al segundo.

- Se ingresan tres notas de un alumno, si el promedio es mayor o igual a siete mostrar un mensaje “Buena nota".

- Se ingresa por teclado un número positivo de uno o dos dígitos (1..99) mostrar un mensaje indicando si el número tiene uno o dos dígitos.
  (Tener en cuenta que condición debe cumplirse para tener dos dígitos, un número entero)

Visual Studio 2022

- Se cargan por teclado tres números distintos. Mostrar por pantalla el mayor de ellos.

- Se ingresa por teclado un valor entero, mostrar una leyenda que indique si el número es positivo, nulo o negativo.

- Confeccionar un programa que permita cargar un número entero positivo de hasta tres cifras y muestre un mensaje indicando si tiene 1, 2, o 3 cifras. Mostrar un mensaje de error si el número de cifras es mayor.

- Un candidato a un empleo, realiza un test de capacitación, se obtuvo la siguiente información: cantidad total de preguntas que se le realizaron y la cantidad de preguntas que contestó correctamente. Se pide confeccionar un programa que permita introducir los dos datos y que informe el nivel del empleado según el porcentaje de respuestas correctas que ha obtenido, y sabiendo que:
Nivel máximo: Porcentaje>=90%. Nivel medio: Porcentaje>=75% y <90%. Nivel regular: Porcentaje>=50% y <75%. Fuera de nivel: Porcentaje<50%.

Variable (por Valor , por Referencia)

- Por **valor**: cuando la variable contiene realmente los datos. Un tipo de valor almacena su contenido en la memoria asignada en la pila. Cuando la variable x queda fuera de ámbito, porque en el método en que se definió ha finalizado la ejecución, el valor se descarta de la pila. Utilizar la pila es eficaz, pero el período limitado de duración de los tipos de valores los hace menos adecuados para compartir los datos entre las diferentes clases. Cuando los datos se pasan a los métodos como parámetros de tipo de valor, se crea una copia de cada parámetro en la pila. Evidentemente, si el parámetro en cuestión es un tipo de datos grande, por ejemplo, una estructura definida por el usuario con muchos elementos, o el método se ejecuta muchas veces, puede afectar el rendimiento.
- Por **referencia**: la variable contiene la dirección de la memoria donde se encuentran los datos. Se asigna en un área diferente de memoria llamada “montón”. En el ejemplo siguiente, el espacio requerido para los diez enteros que constituyen la matriz se asigna en el montón. Hay una sobrecarga mayor al declarar los tipos de referencia, pero éstos tienen la ventaja de ser accesibles desde otras clases.

Variables (por Valor , por Referencia)

```
Int x = 42;       // Por valor
Int[] numero = new int[10];   // Por referencia

Int SumaDiez(int number) // Por valor
{
        return number + 10;
}
Void SumaDiez(ref int number)  // Por referencia
{
        number += 10;
}
```

Constantes

- Valores numéricos o cadenas que no serán modificados durante el funcionamiento de la aplicación

```
const int Valormax = 100;
const string Mensaje = “muy grande”;
const int mitad = Valormax / 2;
…
If (resultado&gt;Valormax)
        Console.writeline(Mensaje);
```

Enumeraciones - constantes

```
enum DiaSemana
{Lunes,Martes,Miercoles,Jueves,Viernes,Sabado,Domingo}
// Interrumpir la secuencia de incremento automático
enum DiaSemanaOtro
{Lunes=10,Martes=20,Miercoles=30,Jueves=40,Viernes=50,
Sabado=60,Domingo=70}
// Siempre son números enteros
// Se pueden utilizar como un tipo
DiaSemanaOtro VarDia;
VarDia = DiaSemana.Lunes;
```

Visual Studio 2022 - Operadores

- **De asignación** =
- **Operadores matemáticos**

  \+ suma

  ++x x++ (+ 1)

  \- resta

  * multiplicación

  / división

  % resto de una división
- **Concatenación** (cadena + “456”);

Matrices

- **Matrices de una dimensión** La creación se efectúa en 2 etapas. Primero declaramos la variable tipo matriz y luego reservamos la memoria para almacenar el contenido de la matriz.

```
//Si queremos guardar la facturación de 12 meses en una matriz
int[] facturación;
facturación = new int[12];
facturación[0]=1500;
facturación[1]=2500;

//Matriz inicializada y donde no es necesario precisar el tamaño de la matriz
double[] tasaIva ={0,4,10,21};
```

Matrices

- Matrices de varias dimensiones

```
int[,,] Cubo;
Cubo = new int[4,4,4];
Cubo[0,1,1] = 52;

//Matriz inicializada
int[,] rejilla2D = {{1,2},{3,4}};
//Obtendremos ahora el tamaño de la matriz, es decir, número de casillas
Console.WriteLine(rejilla2D.Lenght);

int[,] matriz ={{1,2},{3,4},{5,6}};
Console.WriteLine ("Tamaño de la primera dimensión {0}", matriz.GetLenght(0));
Console.WriteLine ("Tamaño de la segunda dimensión {0}", matriz.GetLenght(1));
//Dará 3 y 2 respectivamente

//Si queremos saber las dimensiones de la matriz (2 en nuestro ejemplo)
Console.WriteLine("La dimesión es: {0}", matriz.Rank);
```

Matrices - operaciones

- Búsqueda

```
string[] merienda={"pan","nocilla","mermelada","mantequilla","pate"}
Console.WriteLine(Array.IndexOf(merienda,"mantequilla"));
```

- Ordenar

```
//Ordena alfabéticamente
Array.Sort(merienda);
foreach (string plato in merienda) {
    Console.WriteLine(plato); }
```

Arrays propiedades

- Array.Length Obtiene el número total de elementos de todas las dimensiones de Array.

  Número total de elementos en todas las dimensiones de Array; es cero si no hay elementos en la matriz.

- Array.Rank Obtiene el rango (número de dimensiones) de Array.

  Una matriz unidimensional devuelve 1, una matriz bidimensional devuelve 2, y así sucesivamente.

Arrays métodos

- Array.BinarySearch(Array array, object value) ‘ Array previamente ordenado, busca value en el array
- Array.Clear(array, index, length) Establece un intervalo de elementos de una matriz en el valor predeterminado de cada tipo de elemento.
- Int[] arrClone = arr.Clone(); Crea una copia superficial de la colección array
- Array.Copy(arrOrigen,arrDestino, elemetos) Copia un intervalo de elementos de una matriz Array a otra Array
- Array.GetLowerBound(dimensión) Obtiene el índice del primer elemento de la dimensión especificada en la matriz.
- Array.GetUpperBound(dimensión)Obtiene el índice del último elemento de la dimensión especificada en la matriz.

Arrays métodos

- Array.IndexOf (Array array, object value);Busca el objeto especificado y devuelve el índice de la primera aparición en una matriz unidimensional.
- Array.LastIndexOf (Array array, object value) Devuelve el índice de la última aparición de un valor en una matriz unidimensional
- Array.Resize(ref myArr, myArr.Length + 5); Cambia el número de elementos de una matriz unidimensional al nuevo tamaño especificado.
- Array.Reverse(Array) Invierte el orden de los elementos de un array unidimensional
- Array.Sort(miArr) Ordena los elementos en una matriz unidimensional.

Estructuras

- Posibilidad de crear un nuevo tipo a partir de la combinación de diferentes tipos.

```
Struct Direccion {
        public int codPostal;
        public string calle;
        public string ciudad;
        public string getDireccion() {
                return calle + “\r\n ” + codigoPostal + “\t” +
ciudad.Toupper();
        }
}
  Struct Cliente {
        public int codigo;
        public string apellido;
              public string nombre;
              public string email;
              public Dirección correoPostal ; }
```

Estructuras

```
Cliente unCliente;
unCliente.codigo = 9999;
unCliente.apellido = “García”;
unCliente.nombre = “Juan Carlos”;
unCliente.correoPostal.codPostal=3740;
unCliente.correoPostal.calle =“C\Perico, 3 3º”;
unCliente.correoPostal.ciudad=“Gatos City”
Console.Writeline(uncliente.correoPostal.getDireccion());)
```

Visual Studio 2022 - Operadores

- **Operadores de comparación:**

  > mayor

  < menor

  >= mayor o igual

  <= menor o igual

  == igual

  != distinto

  is compara el tipo de variable con el tipo dado

  ```
  if (edad is int ) { … }
  ```

Visual Studio 2022 – Op. lógicos

Operador	Operación	Ejemplo	Resultado
&	Y lógico	If (test1)&(test2)	Cierto si ambos son ciertos
|	O lógico	If (test1) | (test2)	Cierto si alguno de los dos es cierto
^	O exclusivo	If (test1)^(test2)	Cierto si alguno es cierto y el otro NO lo es
!	Negación	If ! test	Invierte el resultado de test
&&	Y lógico	If (test1)&&(test2)	Igual que &, pero solo evalua test2 si test1 es cierto
||	O lógico	If (test1)||(test2)	Igual que |, pero solo evalua test2 si test1 es falso

Uso de las variables booleanas con los operadores lógicos.

B1	**B2**	**B1 & B2**	**B1 | B2**
null	null	null	null
null	true	null	true
null	false	false	null
true	null	null	true
true	true	true	true
true	false	false	true
false	null	false	null
false	true	false	true
false	false	false	false

Visual Studio 2022

- **Condicional simple:**

```
If (condicion)
{
        resultado verdadero
}
```

- **Condicional compuesta:**

```
If (condicion)
{
        resultado verdadero
}
else
{
        resultado falso
}
```

- **Condicionales anidados**

Visual Studio 2022- cadenas, comparación

```
if (cadena1 == cadena2) {
   Console.WriteLine("son la misma"); }
else {
   Console.WriteLine("no son la misma");}

// Otra forma
if (cadena1.Equals(cadena2)) {
   Console.WriteLine("son la misma"); }
else {
   Console.WriteLine("no son la misma");

// Utilizando CompareTo
switch (cadena1.CompareTo(cadena2)) {
   case  -1:
      Console.WriteLine("cadena1 es inferior a cadena2");
      break;
   case 0:
```

Estructura de control: switch

- Switch (variable) {
  case valor1: …
  break;
  case valor2: …
  break;
  ….
  default: ….
  break;
  }

Estructura repetitiva: while

- La prueba de condición antes de cada repetición, que motivará que se repitan o no las instrucciones.
- While (condicion) { … }
- Do { … } while (condicion);

EJEMPLO:

```
X=1; suma = 0;
While (x<=10)
{
Console.Write (“ Valor ??”);
Valor = int.Parse(Console.ReadLine();
Suma = suma + valor;
X = x + 1;
}
```

Estructura repetitiva: do .. while

```
Do
{
    Console.Write (“Valor: ”);
    Valor = int.Parse(Console.ReadLine());
    If (valor >=100)
      {------}
}
while (valor !=0);
```

Estructuras repetitivas

EJERCICIOS:

- Visualizar valores desde valor inicial al final
- Un bco quiere conocer de cada c/c de sus clientes, titular, nº cta y saldo. Se deben guardar datos de clientes mientras saldo sea = 0 o positivo. Al finalizar mostar un listado con el nombre del titular y estado de la cuenta, acreedor (saldo>0) deudor (saldo <0), nulo(saldo=0) ✉después sacar dinero por cliente

Estructura repetitiva: FOR

- For (valorini del contador ; condicion; instrucción de iteración) { … }

```
Int k1;

For (k1=1; k1 < 10; k1++) {
        for (int k2=1; k2 < 10; k2++) {
                console.write (k1 * k2 + "\t"); }
        console.writeline();
}
```

Estructura de control: foreach

Foreach (elemento en matriz) { … }

```
// FOR
    string [] matriz={“rojo”,”verde”,”azul”,”blanco”};
    int contador;
    for (contador=0; contador < matriz.length; contador ++){
            Console.writeline(matriz[contador])  } ;
// FOREACH
    Foreach (string s in matriz) {
            consle.writeline(s); }
```

Estructura de control: using

- Esta destinada a simplificar el desarrollo. Permite que en un bloque de código podamos utilizar un recurso externo, en el ejemplo, estamos utilizando los recursos que se encuentran en System.IO.StreamReader.

```
Using (System.IO.StreamReader sr = new System.IO.StreamReader
(@"c:\Users\Public\Documents\texto.txt"))
    {
            String s = null;
            While (( s= sr.ReadLine()) != null)
            {
                    Console.WriteLine(s);
            }
    }
```

Funciones y procedimientos

- Todas las instrucciones deben estar incluidas en un procedimiento o función, a las que llamaremos mediante su identificador. A estas funciones y procedimientos podemos pasarles parámetros.
- En **C#** tenemos 4 tipos:
  - Los procedimientos que ejecutan un código a petición sin devolver ningún resultado.
  - Las funciones que ejecutan un código y devuelven el resultado al código que las llamó.
  - Los procedimientos de propiedades que permiten manejar las propiedades de los objetos creados.
  - Los procedimientos de operador utilizados para modificar el funcionamiento de un operador cuando se aplica a una clase o una estructura.

Procedimiento

- **Procedimiento**

  La visibilidad de un procedimiento viene determinada por la declaración ***private***, ***public*** o ***internal***. Por defecto si no se indica nada se entiende que es public.

```
void VerResultado() {
  Console.WriteLine("¡¡¡Ganador!!!");
}
```

Función

- La función devuelve un resultado al código invocante. La ejecución de ***return*** provoca la salida de la función.

```
int calculo () {
  ...
  instrucciones
  ...
  return resultado;
}
```

Procedimiento de propiedades

- Estos procedimientos se llaman “encapsuladores” ya que el valor de la propiedad se encapsula. Se utilizarán cuando queramos modificar y/o recuperar un valor (Set / Get).

```
public tipoDeLa Propiedad nombrePropiedad {
  get {
    ...
    //código que se ejecuta cuando se lee la propiedad
    ...
    return variable;
  }
  set {
    ...
    //código que se ejecuta durante la asignación de una propiedad
    //existe una variable que se declara implícitamente y que contiene
    //el valor que se debe asignar a la propiedad
    ...
    variable = value;
    ...
  }
}
```

Si una propiedad es de sólo lectura o sólo escritura, se eliminará el

Procedimiento de operador

- Permite la redifinicón de un operador estándar del lenguaje para utilizarlo en tipo personalizados (clase o estructura).

```
struct Cliente {
  public int codigo;
  public string apellido;
  public string nombre;
}

Cliente c1, c2, c3;
  c1.codigo = 200;
  c1.nombre = "Juanjo";
  c1.apellido = "Pedraza";

  c2.codigo = 125;
  c2.nombre = "Perico";
  c2.apellido = "Palotes";

  c3 = c1 + c2;
  //Aquí el compilador daría error porque no se pueden aplicar el operando al tipo.
}
```

Procedimiento de operador

```
struct Cliente {
  public int codigo;
  public string apellido;
  public string nombre;
  public static Cliente operator + (Cliente cl1, Cliente cl2) {
    Cliente c;
    c.codigo = cl1.codigo + cl2.codigo;
    c.apellido = cl1.apellido + cl2.apellido;
    c.nombre = cl1.nombre + cla2.nombre;
    return c;
  }
}
```

Sobreescribir método

```
public override string ToString()
    {
        return "Name = " + Name + ", Age = " + Age;
        }
```

Argumentos de procedimientos – por valor

En este caso PrecioBruto se está pasando por valor ya que de forma predeterminada los valores enteros, números flotantes, decimales, booleanos y estructuras definidas por el usuario se pasan por valor. Los demás tipos siempre se pasan por referencia, es decir se le pasa la dirección de memoria a la que apunta la variable de forma que el procedimiento o función pueden manipular directamente el valor de esa variable.

```
public static double CalculoNETO (double Pbruto, double Tasa) {
  return Pbruto * (1+(Tasa/100));
}
...
double PrecioBruto = 100;
double PrecioNeto;
PrecioNeto = TestEstructura.CalculoNETO(PrecioBruto, 5.5);
Console.WriteLine(PrecioNeto);
```

Argumentos de procedimientos – por referencia

Podemos forzar el paso por referencia de una variable que por defecto se pasa como valor. para ello utilizaremos la palabra reservada "**ref**" o "**out**", la etiqueta "**ref**" se debe utilizar tanto en la lista de parámetros de la función o procedimiento, como en la propia llamada a la función o procedimiento y además debe ser inicializada, por el contrario, la etiqueta "**out**" funciona de igual manera pero sin la exigencia de inicializar la variable.

```
public static double CalculoNETO (double Pbruto, double Tasa, ref double iva) {
  iva = Pbruto * (Tasa/100);
  return Pbruto+iva; }


...
double PrecioBruto = 100;
double PrecioNeto;
double importeIva=0;
PrecioNeto = TestEstructura.CalculoNETO(PrecioBruto, 5.5, ref importeIVA);
Console.WriteLine("Precio neto: {0}", PrecioNeto);
Console.WriteLine("Importe iva: {0}", importeIva);
```

Argumentos de procedimientos – por referencia

- Ejemplo de procedimiento con nº de parámetros indeterminado

```
public staic double media(params double[] notas)
{
  double suma=0;
  foreach (double nota in notas)
            {
            suma = suma + nota;
            }
  return suma /notas.Length;
}
```

```
Resultado = media(1,4,67,233);
Resultado = media(23,33);
```

Parámetros opcionales

- Se puede indicar que un parámetro es opcional asignándole un valor, pero con la precaución de asignar también a todos los parámetros restantes un valor ya que al declarar un parámetro como opcional, el resto de parámetros también deberían serlo.

  ```
  double calculoNeto(double Pbruto, double Tasa = 21);
  ```

- La siguiente declaración estaría mal ya que el último parámetro no sería opcional:

  ```
  double calculoNeto(double Pbruto, double Tasa = 21; String divisa);
  ```

- Lo correcto sería

  ```
  double calculoNeto(double Pbruto, double Tasa = 21; String divisa="€");
  ```

- Por otro lado en la llamada a la función si se especifica un parámetro opcional, todos los anteriores también se deberían definir:

  ```
  calculoNeto(10);
  calculoNeto(10,5.5);
  calculoNeto(10,5.5,"$");
  ```

- En cambio esto daría error: `calculoNeto(10, ,"$");`

Parámetros nominados

- Podemos hacer la llamada a la función sin tener en cuenta el orden siempre que nominemos los parámetros. Por ejemplo:

```
calculoNeto(divisa:"$",Pbruto:250);
```

- incluso podemos combinar los parámetros por posición y por nominación

```
calculoNeto(10, divisa: "$");
```

- pero hay que seguir la regla de que un parámetro nominado sólo puede ser utilizado después de los parámetros por posición, el siguiente ejemplo fallaría:

```
//daría error
calculoNeto(10,Tasa: 5.5 ,"$");
```

Clase predefinidas

- MessageBox.Show(“Aviso error”);
- MessageBox.Show (Msg,Tituto,MessageBoxButtons.YesNo,
  MessageBoxIcon.Question,
  MessageBoxDefaultButton.Button2,
  MessageBoxOptions.xxx)
- DialogResult result
  =MessageBox.Show(“x”,”y”,MessageBoxButtons.YesNoCancel)
  If (result = DialogResult.Yes) { .. Instrucciones ..}

Clase MessageBox

- **MessageBoxButtons**. [AbortRetryIgnore| Ok | OkCancel | RetryCancel | YesNo |YesNoCancel
- **MessageBoxIcon**. [Asterisk|Error|Exclamation|Hand| Information|None|Question|Stop|Warning]
- **MessageBoxDefaultButton**.[Button1,Button2,Button3]
- **MessageBoxOptions**[RightAlign,Rtlreading,ServiceNotification]
- **DialogResult** [None|OK|Cancel|Abort|Retry|Ignore|Yes|No]

Visual Studio 2022

- Realizar un programa que pida cargar una fecha cualquiera, luego verificar si dicha fecha corresponde a Navidad.

- Se ingresan tres valores por teclado, si todos son iguales se imprime la suma del primero con el segundo y a este resultado se lo multiplica por el tercero.

- Se ingresan por teclado tres números, si todos los valores ingresados son menores a 10, imprimir en pantalla la leyenda "Todos los números son menores a diez".

- Se ingresan por teclado tres números, si al menos uno de los valores ingresados es menor a 10, imprimir en pantalla la leyenda "Alguno de los números es menor a diez".

- De un operario se conoce su sueldo y los años de antigüedad. Se pide confeccionar un programa que lea los datos de entrada e informe:
  a) Si el sueldo es inferior a 500 y su antigüedad es igual o superior a 10 años, otorgarle un aumento del 20 %, mostrar el sueldo a pagar.
  b)Si el sueldo es inferior a 500 pero su antigüedad es menor a 10 años, otorgarle un aumento de 5 %.
  c) Si el sueldo es mayor o igual a 500 mostrar el sueldo en pantalla sin cambios.

Visual Studio 2022

- Windows Forms
  → program.cs, punto de entrada principal para la aplicación
  → form1.cs, crea el formulario
  → form1.designer.cs , diseñador windows form
- Cuadro de herramientas
- Ventana propiedades

  .name, .text

Visual Studio 2022

- **Cuadro de herramientas**

  Los componentes visuales, agrupados por su uso

Visual Studio 2022

- **Ventana de propiedades**

  permite inicializar los valores de las propiedades del objeto que se encuentra seleccionado en el formulario (Button, MonthCalendar, TextBox etc.)

Visual Studio 2022

- **Ventana de eventos**

  La ventana de eventos coincide con la ventana de propiedades.

  Para disponer el código para dicho evento debemos hacer doble clic sobre dicho evento (esto hace que se active la ventana del editor y genere automáticamente el método asociado a dicho evento)

  Disponer 7 objetos de la clase Button con los días de la semana. Fijar en los atributos Text de cada botón los días de la semana. Al presionar un botón mostrar en un objeto de la clase Label el día seleccionado.

Visual Studio 2022

- Control común – Label

  permite mostrar un texto inicializando la propiedad Text.

  Text
  BackColor
  ForeColor
  Font
  BorderStyle
  AutoSize
  Cursor
  Visible
  Enabled

Visual Studio 2022

- Controles comunes – Button

  (1) Confeccionar un formulario que muestre tres objetos de la clase Button, disponer como etiqueta en cada botón los valores 1,2 y 3. Cuando se presiona el botón mostrar en el título del formulario el valor de la etiqueta del botón presionado.

  (2) Modificar el problema anterior para que se acumulen en el título del formulario los valores de los botones presionados. + , &

  (3) Similar al problema anterior solo permitir mostrar hasta 10 caracteres en el título del formulario.

  (4) Elaborar una interfaz gráfica que muestre una calculadora (utilizar objetos de la clase Button y un objeto de la clase Label donde se muestra el valor ingresado), tener en cuenta que solo se debe implementar la interfaz y la carga de un valor de hasta 12 dígitos.